Tom Mitchell

chapter: 01

learning problem: -

A computer program is said to be learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

- well defined learning problem have 3 features:

the class of task, the measure of performance to be improved, and the source of experience.

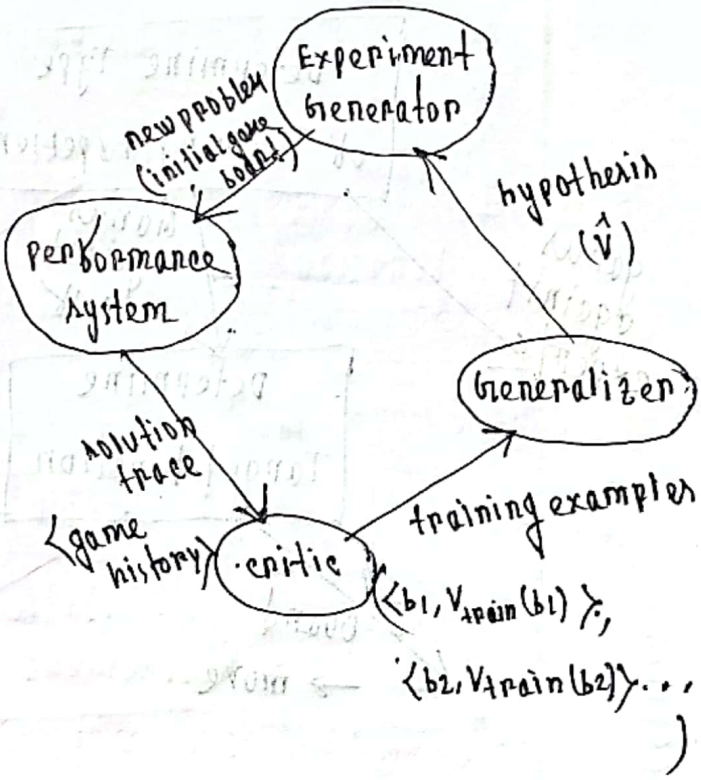
some successful applications of ML:

- learning to recognize spoken words.
- learning to drive an autonomous vehicle
- learning to classify new astronomial structures
- learning to play world class backgammon

- A checker learning problem:
  - Task T: playing checkers
  - performance measure, P: percent of games won against opponent
  - Training experience, E: playing practise problem itself
- A handwriting recognition learning problem:
  - Task T: recognizing and classifying handwritten words within images.
  - performance measure, P: percen of words correctly classified.
  - Training experience, E: a database of handwritten words with given classification.
- A robot driving learning problem:
  - Task T: driving on public four-lane highways using vision sensors.
  - performance measure P: distance travelled before an error
  - Training experience E: a sequence of images and steering commands recorded while observing a human driver.

Final design of the checker learning program

- performance system in the module must solve the given performance task, in this case playing checkers, by using the learned target function.

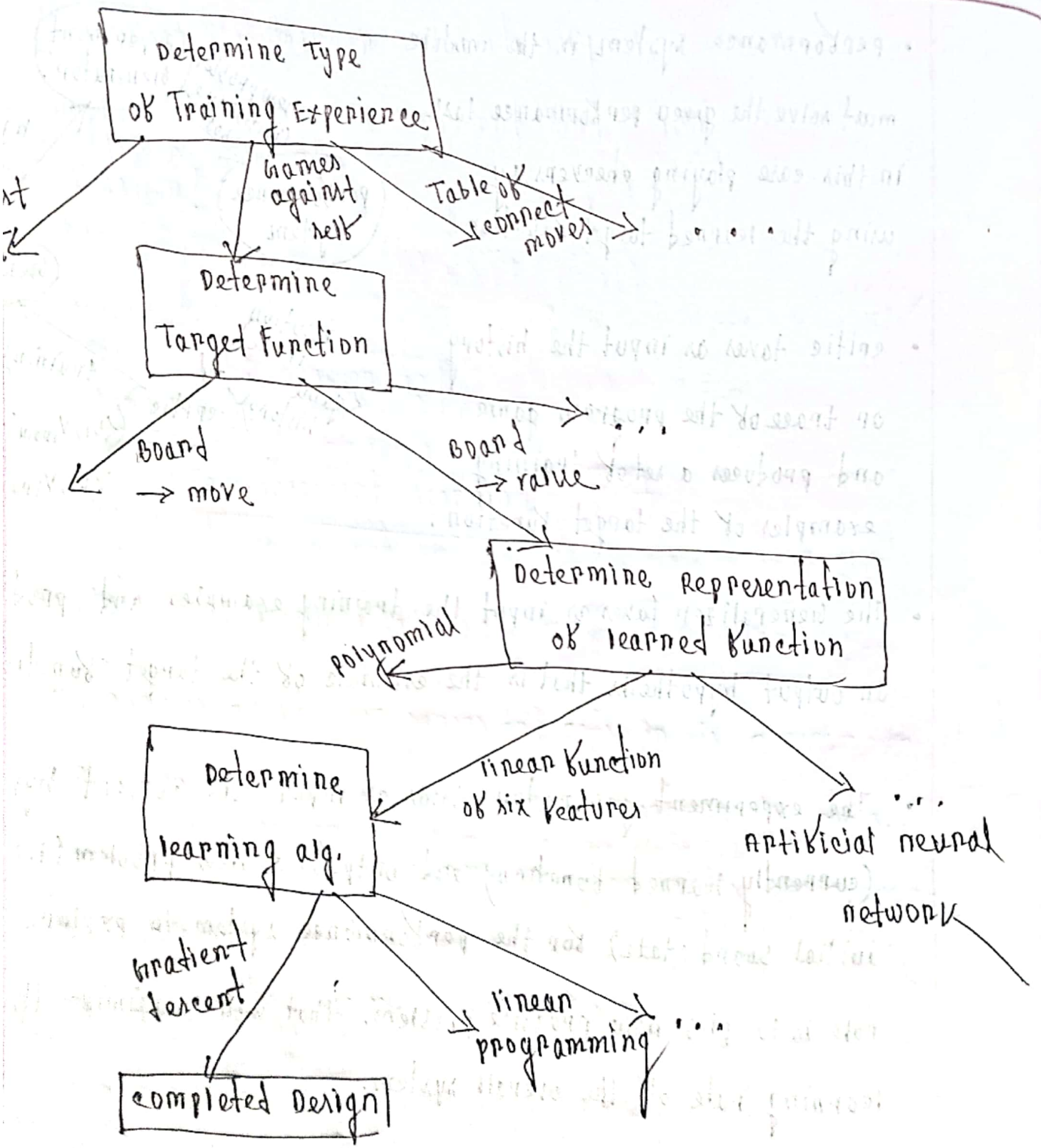


- critic takes as input the history or trace of the program game and produces a set of training examples of the target function.

- The generalizer takes as input the training examples and produces an output hypothesis that is the estimate of the target function.

- The experiment generator takes as input the current hypothesis (currently learned function) and outputs a new problem (i.e. initial board state) for the performance system to explore. Its role is to pick new practise problems that will maximize the learning rate of the overall system.

summary of choices in designing the checker learning program



1. what algorithms exist for learning general target functions from specific training examples? in what setting will particular algs converge to the desired function, given sufficient training data? which algs perform best for which types of problems and representations?

2. How much training data is sufficient? what general bounds can be found to relate the confidence in learned hypotheses to the amount of training experience and the character of the learner's hypothesis space?

3. when and how can prior knowledge held by the learner guide the process of generalizing from examples? can prior knowledge be helpful even when it is only approximately correct?

4. what is the best strategy for choosing a useful next training experience, and how does the choice of this strategy alter the complexity of the learning problem?

5. what is the best way to reduce the learning task to one or more function approximation problems? put another way, what specific functions should the system attempt to learn? can this process itself be automated.

6. How can the learner automatically alter its representation to improve its ability to represent and learn the target function?

: chapter - 13

Reinforcement learning

Reinforcement learning addresses the question of how an autonomous agent that senses and acts in its environment can learn to choose optimal actions to achieve its goals. Each time the agent performs an action in its environment, a trainer may provide a reward or penalty to indicate the desirability of the resulting state.

For example,

when training an agent to play a game, the trainer might

provide a positive reward when the game is won, negative reward when it is lost, and zero rewards in all other states. The task of the agent is to learn from this indirect, delayed reward, to choose sequences of actions that produce the greatest cumulative award.

- the RL problem differs from other function approximation tasks in several important respects:

1. Delayed Reward: After action ~ reward assign
2. Exploration: unknown states & actions OR knowns
   - (to gather new info) ↓
   - (to max. cumulative reward) ↓
3. partially observable states (not fully observable)
4. like long learning: learn several related tasks within the same environment ~ uses previously obtained experience or knowledge to reduce sample complexity when learning new tasks.

Q learning:—

- the Q function:

$$Q(s,a) \equiv r(s,a) + \gamma V^*(\delta(s,a))$$

(plus the value discounted by γ)

- $Q(s,a)$: evaluation function
- $r(s,a)$: reward received after performing action a.
- $0 \leq \gamma < 1$: discount constant

- Algorithm:

For each s,a initialize the table entry $\hat{Q}(s,a)$ to zero

Observe the current state s

Do forever:

- select an action a and execute it
- Receive immediate reward r
- observe the new state s'
- update the table entry for $\hat{Q}(s,a)$ as follows:

$$\hat{Q}(s,a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s',a')$$

- $s \leftarrow s'$

Q learning algorithm, assuming rewards and actions.

- kernel functions:

The idea of the kernel function is to enable operations to be performed in the input space rather than the potentially high dimensional feature space. Hence the inner product doesn't need to be evaluated in the feature space. The kernel function plays a critical role in svm and its performance. It is based upon reproducing kernel Hilbert spaces:

$$K(x, x') = (\phi(x), \phi(x'))$$

Different types of kernel function are listed below:

1. polynomial: * popular method for non-linear modeling

$$K(x, x') = (x, x')^d$$

$$K(x, x') = ((x, x') + 1)^d \rightarrow \text{preferable as it avoids problems with the hessian becoming zero.}$$

2. Gaussian Radial Basis Function:

Radial basis functions most commonly with a gaussian form,

$$k(x,x') = \exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$$

3. Exponential Radial Basis Function:

A radial basis function produce a piecewise linear solution which can be attractive when discontinuities are acceptable.

$$k(x,x') = \exp\left(-\frac{\|x-x'\|}{2\sigma^2}\right)$$

4. multi-layer perceptron:

The long established MLP, with a single hidden layer, also has a valid kernel representation.

$$k(x,x') = \tanh\left(p(x,x') + e\right)$$

Others: Fourier, splines, additive kernels . . .

B-

- controlling complexity in svm

svm is powerful to approximate any training data and generalizes better on given datasets. The complexity in terms of kernel affects the performance on new datasets. svm supports parameters for controlling the complexity and above all, svm doesn't tell us how to set these parameters and we should be able to determine these parameters by cross validation on the given datasets. The diagram given below gives a better illustration.

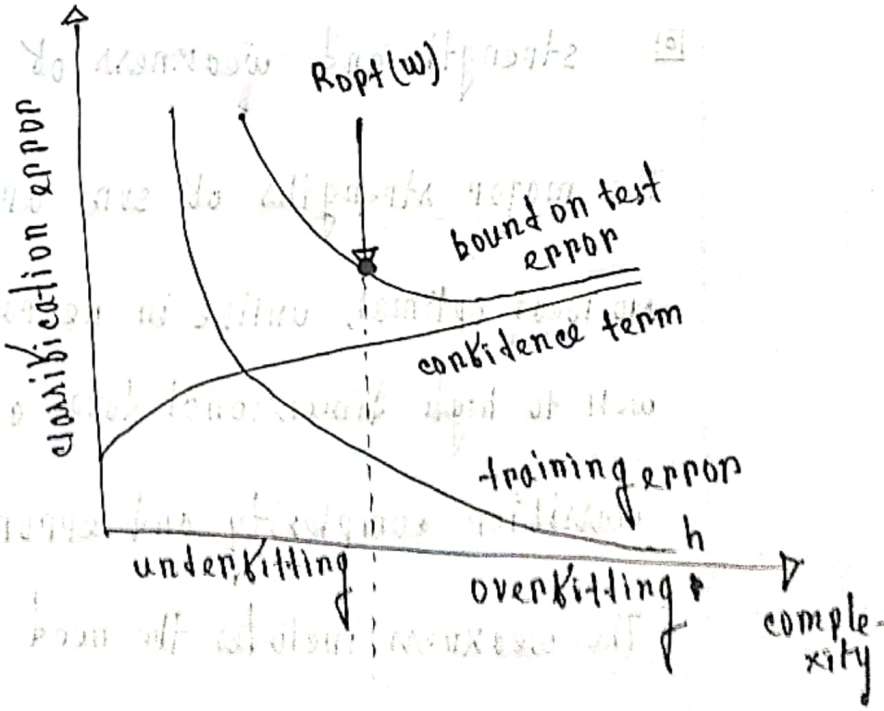


Figure: how to control complexity.

- Application of SVM

✓ Image classification, ✓ Text classification, ✓ Bioinformatics,
(based on their features) (analyze biological data such as DNA and protein sequence, to predict their function)

✓ Handwriting recognition

✓ Finance (predicting stock prices on market based on historical data and other factors)

✓ medical diagnosis

✓ object recognition (in images or videos)

✓ pattern classification

- strength and weakness of SVM:

The major strengths of SVM are the training is relatively easy. No local optimal, unlike in neural network. It scales relatively well to high dimensional data and the trade-off between classifier complexity and error can be controlled explicitly. The weakness includes the need for a good kernel function.